1. What is promise?

A promise is a javascript object that represents the eventual result of an operation that may complete in future.

That result can be:

→ A successful value, such as user data received from an API.
→ A failure, such as a network error.
→ A result that is not ready yet.

features:

→ A promise can be fulfilled & Rejected
→ Promises help managing asynchronous operations
→ promises can be chained using .then().
→ Errors can be handled using .catch()
→ promises are used by APIs such as fetch().

2. What are the three Promise states?

1. pending

The operation is still in progress.
The promises has not yet fulfilled or rejected.

2. fulfilled

The Operation completed successfully.
The promise has a successfull result value

3. Rejected

The operation failed.
The promises has a rejection reason, commonly an error.

3. Difference between resolve & reject?

resolve() and reject() are functions provided to the promise executor.

resolve():

resolve() is used to indicate that an operation completed successfully.

The promise becomes fulfilled. The value can be accessed using .then()

reject():

reject() is used to indicate that an operation failed.

The promise becomes rejected. The rejection can be handled using .catch().

4. What is .then()?

- then() is a promise method used to handle the successful result of a promise.

It can also receive a second callback for handling rejection, although .catch() is more commonly used for errors.

- then() can be chained to perform multiple operations sequentially.

5. What is .catch()?

- catch() is a promise method used to handle rejected promises and errors that occurs during promises execution.

The error parameters receives the rejection reason.

6. What is .finally()?

- finally() is a promise method that executes after a promises settles, whether it is fulfilled or rejected.

Settled means the promises is either:

- Fulfilled
- Rejected

- finally() normally does not receive the fulfillment value or rejection reason as an argument.

It also does not change the promise's outcome unless it throws an error or returns a rejected promises

7. What is fetch()?

fetch() is a build in javascript API used to make HTTP requests to servers. It is commonly used to communicate with backend APIs.

fetch() returns a promise. That promise fulfills with a response object when the browser receives an HTTP response.

8. Why do we use response.json()?

1. Reads the response body.
2. Passes the JSON data.
3. Returns a promise that fulfills with the resulting javascript values

response.json() is appropriate when the response body contains valid JSON.

9. What is async?

async is a keyword used to declare an asynchronous function. An async function always returns a promises.

Async automatically wraps a returned value in a fulfilled promise.

We use async when we want to write asynchronous code in a clean er way, especially when working with await

10. What is await?

await is a keyword used to wait for a promise's result inside an async function.

It makes asynchronous code look more like synchronous code.

await does not blck the entire javascript application.

11. What is the difference between .then() & await?

Both .then() & await are used to handle promises. They can be used to perform the same asynchronous operation, but they are use different syntax.

- then():
  - then() handles the fulfilled promise.
  - receives the response the response object.
  - receives the passed data.
- catch() handles rejected promises and errors in the chain.

Async:

- async declares an async function.
- await fetch() waits for the response promise.
- await response.json() awaits for the passed data.
- try...catch handles errors.

12. Does fetch() reject for a 404 error?

No. fetch() does not automatically reject a promise when the server returns an HTTP error status such as 404, 400 or 500.

Instead fetch(), considers the request successful at the network level if it receives an HTTP response, even if that response contains an error status.

→ javascript sends a request

→ server receives the request

→ server responds with 404.

→ Fetch receives the HTTP response.

Fetch gives a response object, not an automatically rejected promise